Universidad Tecmilenio

Campus Las Torres

Materia: Desarrollo FullStack

Proyecto Final

Alumno: Patricio Calvo

Matricula: 07097795

Febrero / 2026

Link de Github:

https://github.com/Diablo650cc/Desarrollo-FullStack/tree/main/Proyecto%20Final

Link de Video:

https://drive.google.com/file/d/17qw5eXV22B8Q7l39C3zRwqvLXgR6gQ6C/view?usp=drive_link

DOCUMENTACIÓN DEL PROYECTO

Requerimientos Funcionales

Gestión de Usuarios

- Registro de nuevos usuarios (email y contraseña)
- Inicio de sesión con email y contraseña
- Cierre de sesión
- Las contraseñas se guardan encriptadas en la base de datos

Gestión de Productos (CRUD)

- Crear: Agregar nuevos productos
- Leer: Ver lista de todos los productos disponibles
- Actualizar: Modificar datos de un producto existente
- Eliminar: Borrar productos del catálogo

Seguridad

- Solo usuarios autenticados pueden crear, actualizar o eliminar productos
- Usuarios no autenticados solo pueden VER los productos
- Cada usuario solo puede modificar/eliminar sus propios productos

Requerimientos No Funcionales

Técnicos

- Backend: Node.js con Express
- Base de datos: MongoDB
- Autenticación: Tokens JWT
- Pruebas: Jest

Rendimiento

- Tiempo de respuesta < 2 segundos
- API RESTful con códigos HTTP apropiados

Despliegue

- Código en GitHub con Actions para pruebas automáticas
- App desplegada en Vercel

UX/UI

- Mensajes de error claros para el usuario
- Confirmación antes de eliminar productos

CONFIGURACIÓN DEL SERVIDOR

Estructura de Carpetas Creada

```
motogear-crud/
├── src/
│   ├── config/ # Configuración de base de datos
│   ├── models/ # Modelos de Usuario y Producto
│   ├── controllers/ # Lógica de negocio
│   ├── routes/ # Definición de endpoints
│   ├── middlewares/ # Autenticación y protección
│   └── app.js # Configuración de Express
├── .env # Variables de entorno
├── package.json # Dependencias y scripts
└── server.js # Punto de entrada
```

Configuración Inicial

- Inicialización del proyecto con npm init -y.
- Instalación de dependencias principales: express, mongoose, bcryptjs, jsonwebtoken, dotenv, cors.
- Instalación de dependencias de desarrollo: nodemon, jest, supertest.
- Configuración de scripts en package.json (start, dev, test).

```
2   "name": "motogear-crud",
3   "version": "1.0.0",
4   "description": "CRUD de productos de motociclismo con autenticación JWT",
5   "main": "server.js",
    ▷Debug
6   "scripts": {
7     "start": "node server.js",
8     "dev": "nodemon server.js",
9     "test": "jest"
10  },
11  "dependencies": {
12    "express": "^4.18.2",
13    "mongoose": "^7.5.0",
14    "bcryptjs": "^2.4.3",
15    "jsonwebtoken": "^9.0.2",
16    "dotenv": "^16.3.1",
17    "cors": "^2.8.5"
18  },
19  "devDependencies": {
20    "nodemon": "^3.0.1",
21    "jest": "^29.7.0",
22    "supertest": "^6.3.3"
```

Base de Datos (MongoDB)

- Creación de archivo de conexión en src/config/database.js.
- Configuración de conexión usando mongoose.
- Manejo de errores y cierre graceful.

```
ear-crud > src > config > JS database.js > ...
const mongoose = require('mongoose');

const connectDB = async () => {
    try {
        const conn = await mongoose.connect(process.env.MONGODB_URI);
        console.log(`MongoDB conectado: ${conn.connection.host}`);
    } catch (error) {
        console.error('Error conectando a MongoDB:', error.message);
        process.exit(1);
    }
};

module.exports = connectDB;
```

Modelo de Usuario

- Schema con email único y password.
- Encriptación automática de contraseñas con bcryptjs mediante hash y salt.
- Método personalizado para comparar contraseñas.
- Validaciones de campos requeridos.

Modelo de Producto

- Schema con campos: nombre, categoría, precio, talla, color, stock.
- Categorías predefinidas: Cascos, Chaquetas, Guantes, Botas, Accesorios.
- Referencia al usuario creador mediante ObjectId.
- Validaciones de tipos y valores mínimos.

Middleware de Autenticación

- Extracción de token JWT del header Authorization.
- Verificación de token usando jwt.verify.
- Búsqueda del usuario asociado al token.
- Adjuntar usuario al objeto req para uso en rutas protegidas.

```
motogear-crud > src > middlewares > JS authMiddleware.js > checkProductOwner > message
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const Product = require('../models/Product');

const protect = async (req, res, next) => {
    let token;

    if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
        token = req.headers.authorization.split(' ')[1];
    }

    if (!token) {
        return res.status(401).json({ message: 'No autorizado, token requerido' });
    }

    try {
        // Verificar token
        const decoded = jwt.verify(token, process.env.JWT_SECRET);

        // Obtener usuario del token
        req.user = await User.findById(decoded.id).select('-password');

        if (!req.user) {
            return res.status(401).json({ message: 'Usuario no encontrado' });
        }

        next();
    } catch (error) {
        console.error(error);
        return res.status(401).json({ message: 'Token inválido' });
```

Controladores Implementados

- Auth Controller: register para creación de nuevos usuarios con email y password.
- login para verificación de credenciales y generación de token JWT.
- getProfile para obtención de datos del usuario autenticado.
- Product Controller: getProducts para lista pública de todos los productos.
- getProductById para detalle de producto específico.
- createProduct para creación de producto requiriendo autenticación.
- updateProduct para modificación de producto existente.
- deleteProduct para eliminación de producto.

Rutas Configuradas

- Rutas de Autenticación en /api/auth: POST /register para registro de usuario, POST /login para inicio de sesión, GET /profile para perfil protegido.

```
const express = require('express');
const { register, login, getProfile } = require('../controllers/authController');
const { protect } = require('../middlewares/authMiddleware');

const router = express.Router();

router.post('/register', register);
router.post('/login', login);
router.get('/profile', protect, getProfile);

module.exports = router;
```

- Rutas de Productos en /api/products: GET / para listar productos públicos, GET /:id para ver producto público, POST / para crear producto protegido, PUT /:id para actualizar protegido requiriendo propietario, DELETE /:id para eliminar protegido requiriendo propietario.

```
const express = require('express');
const {
    getProducts,
    getProductById,
    createProduct,
    updateProduct,
    deleteProduct
} = require('../controllers/productController');
const { protect } = require('../middlewares/authMiddleware');

const router = express.Router();

// Rutas públicas
router.get('/', getProducts);
router.get('/:id', getProductById);

// Rutas protegidas
router.post('/', protect, createProduct);
router.put('/:id', protect, updateProduct);
router.delete('/:id', protect, deleteProduct);
```

Seguridad Implementada

- Contraseñas encriptadas con bcryptjs.
- Tokens JWT con expiración de 7 días.
- Verificación de propiedad para modificar o eliminar productos.
- Validaciones en todos los endpoints.
- Middleware de protección para rutas sensibles.
- Manejo de errores global.

Variables de Entorno (.env)

- PORT=3000
- MONGODB_URI=mongodb://localhost:27017/motogear
- JWT_SECRET=mi_secreto_super_seguro_cambiame_en_produccion
- JWT_EXPIRE=7d

Ejecución del Proyecto

- Servidor iniciado con npm run dev usando nodemon para auto-reload.
- Conexión exitosa a MongoDB local. API disponible en http://localhost:3000

```
C:\WINDOWS\system32\cmd.

Microsoft Windows [Version 10.0.26100.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\PatoCV\Desktop\ACT 4\motogear-crud>npm run dev

> motogear-crud@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.3.1] injecting env (4) from .env -- tip: 🛡️ auth for agents:
https://vestauth.com
Servidor corriendo en puerto 3000
```

GENERACIÓN DE RUTAS

Añadimos y ajustamos los middlewares de authMiddleware.js:

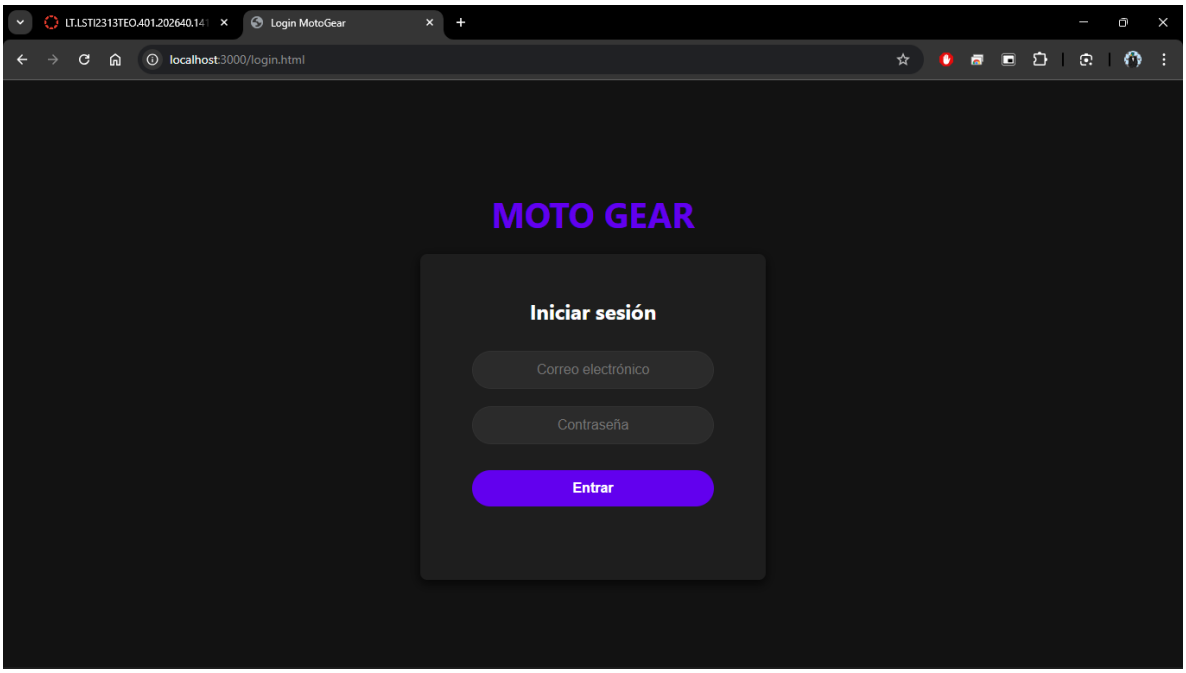
- protect valida token JWT y carga req.user.
- checkProductOwner deja pasar sólo al creador o a un admin.
- admin restringe rutas a administradores.

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const Product = require('../models/Product');

const protect = async (req, res, next) => {
    let token;

    if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
        token = req.headers.authorization.split(' ')[1];
    }

    if (!token) {
        return res.status(401).json({ message: 'No autorizado, token requerido' });
    }

    try {
        // Verificar token
        const decoded = jwt.verify(token, process.env.JWT_SECRET);

        // Obtener usuario del token
        req.user = await User.findById(decoded.id).select('-password');

        if (!req.user) {
            return res.status(401).json({ message: 'Usuario no encontrado' });
```

```
// Middleware para verificar propiedad del producto
const checkProductOwner = async (req, res, next) => {
    try {
        const product = await Product.findById(req.params.id);

        if (!product) {
            return res.status(404).json({ message: 'Producto no encontrado' });
        }

        // permitir al propietario o a un administrador
        if (
            product.usuario.toString() !== req.user._id.toString() &&
            req.user.role !== 'admin'
        ) {
            return res.status(403).json({ message: 'No tienes permiso para modificar este producto' });
        }

        req.product = product;
        next();
    } catch (error) {
        res.status(500).json({ message: 'Error verificando propietario' });
    }
};

// middleware para rutas sólo accesibles por administradores
const admin = (req, res, next) => {
    if (!req.user || req.user.role !== 'admin') {
        return res.status(403).json({ message: 'Se requiere rol de administrador' });
```

IMPLEMENTACIÓN DE AUTENTICACIÓN Y AUTORIZACIÓN

- Se añadió JWT en authController y protect en el middleware para validar el token.
- Las rutas de productos (productRoutes.js) usan protect y checkProductOwner para asegurar creación/edición/borrado.
- Se sirvió un HTML estático (public/login.html) con un formulario que pide email/clave y guarda el token.
- app.js expone la carpeta public/ para que login.html sea accesible en /login.html.



GENERACIÓN DE CONTROLADORES

Modelos Mongoose:

- User.js: Esquema con email (único), contraseña (encriptada con bcrypt), timestamps. Método pre-save para encriptación y método comparePassword().
- Product.js: Esquema con nombre, categoría (enum), precio, talla, color, stock, y relación con usuário via ObjectId.
- Controladores:

AuthController.js:

- register(): User.findOne() para verificar existencia, User.create() para crear usuario
- login(): User.findOne() para buscar, comparePassword() método del modelo
- getProfile(): Retorna datos del usuario autenticado

GENERACIÓN DE PRUEBAS UNITARIAS

Configuración Jest:

- jest.config.js (test environment, coverage paths)
- jest.setup.js (timeout global)
- .env.test (variables de entorno)
- Scripts NPM agregados (test, test:watch, test:coverage, etc.)

60 Tests Creados:

- 28 tests - Controladores (auth + products)
- 10 tests - Middlewares (protección)
- 22 tests - Rutas/Integración HTTP

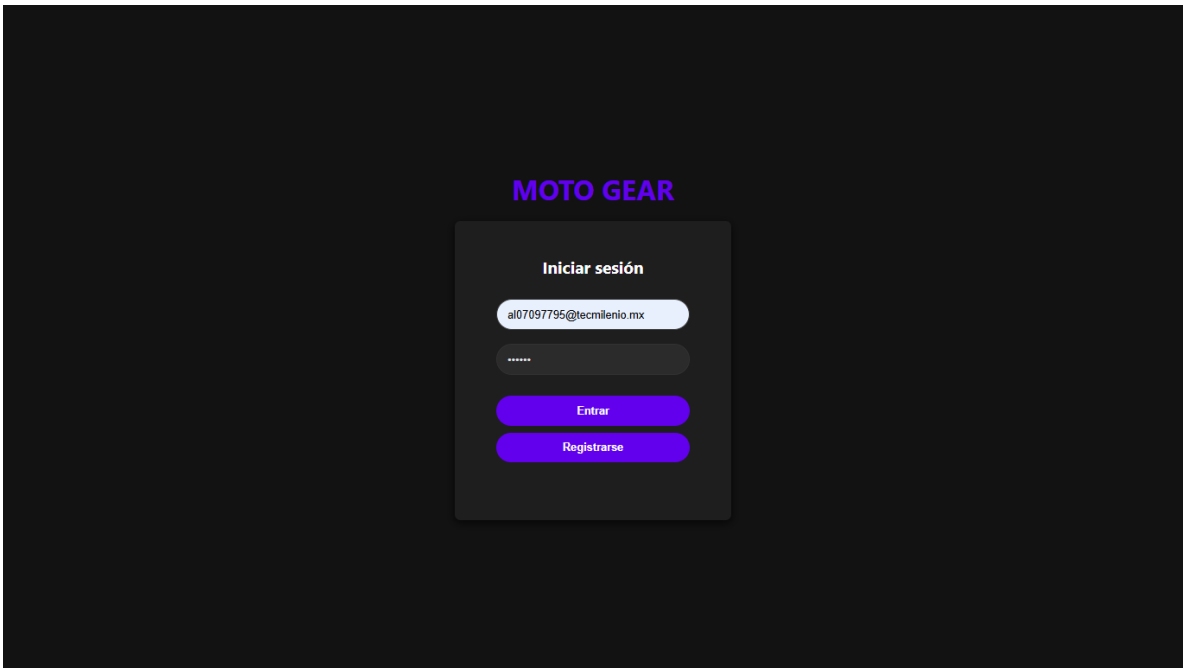
Validaciones Cubiertas:

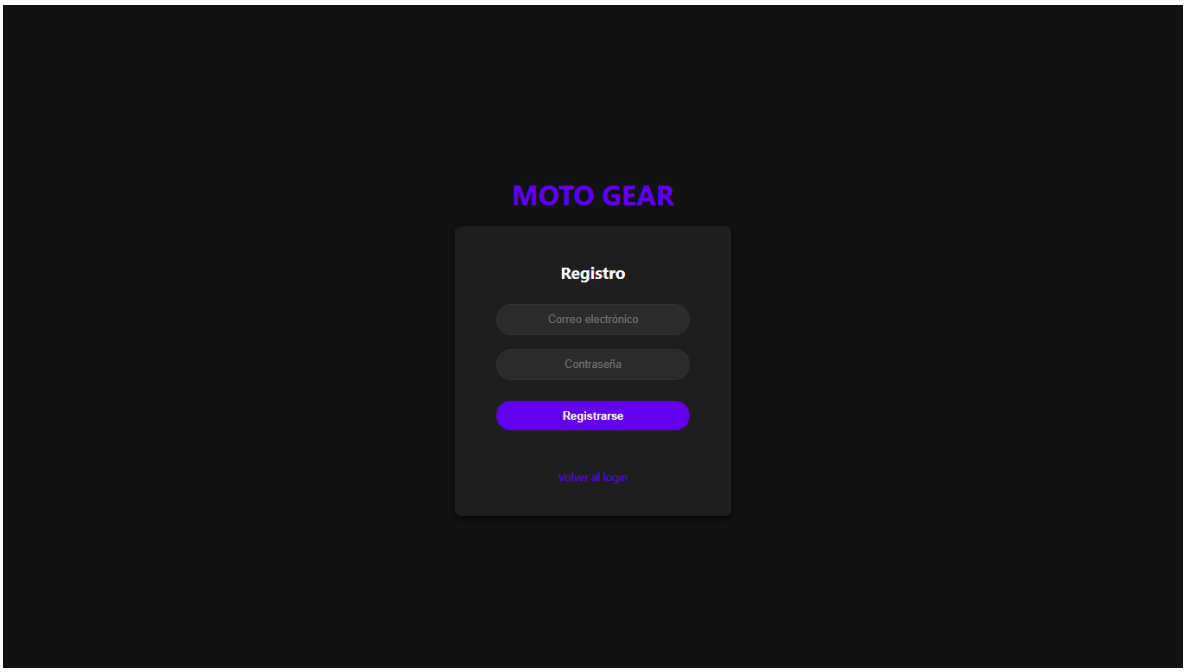
- Registro y login usuarios
- CRUD completo productos
- Autenticación JWT
- Autorización/propiedad
- Errores HTTP (400, 401, 403, 404, 500)
- Validación de campos

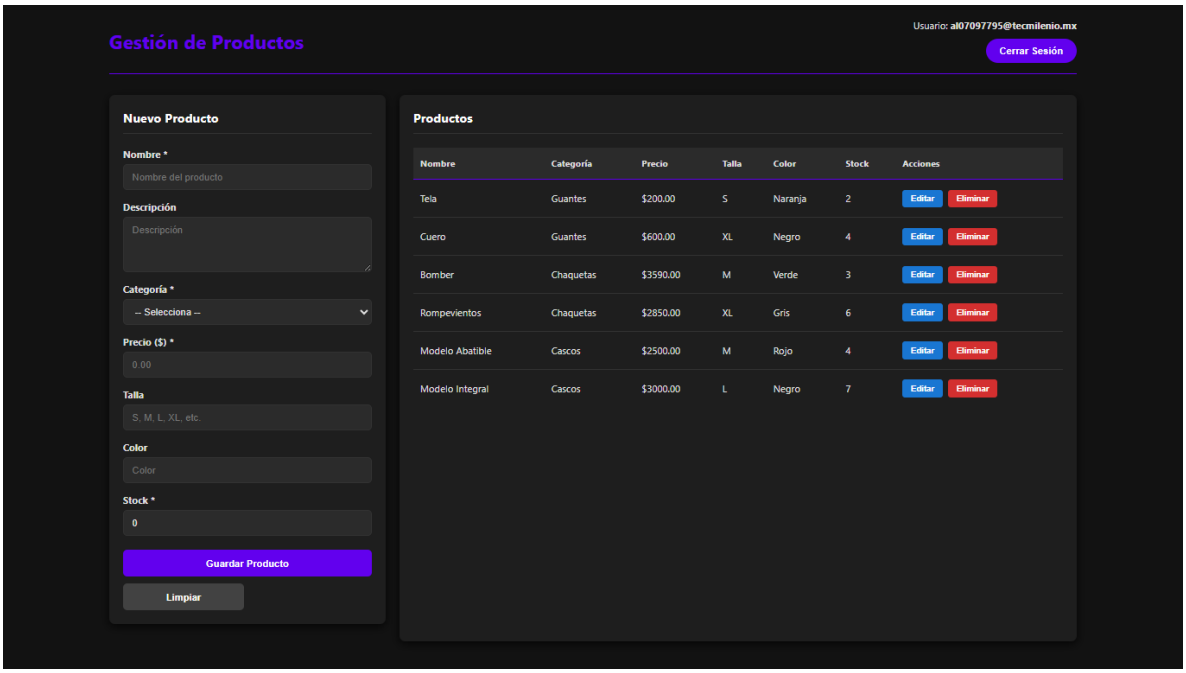
Documentación:

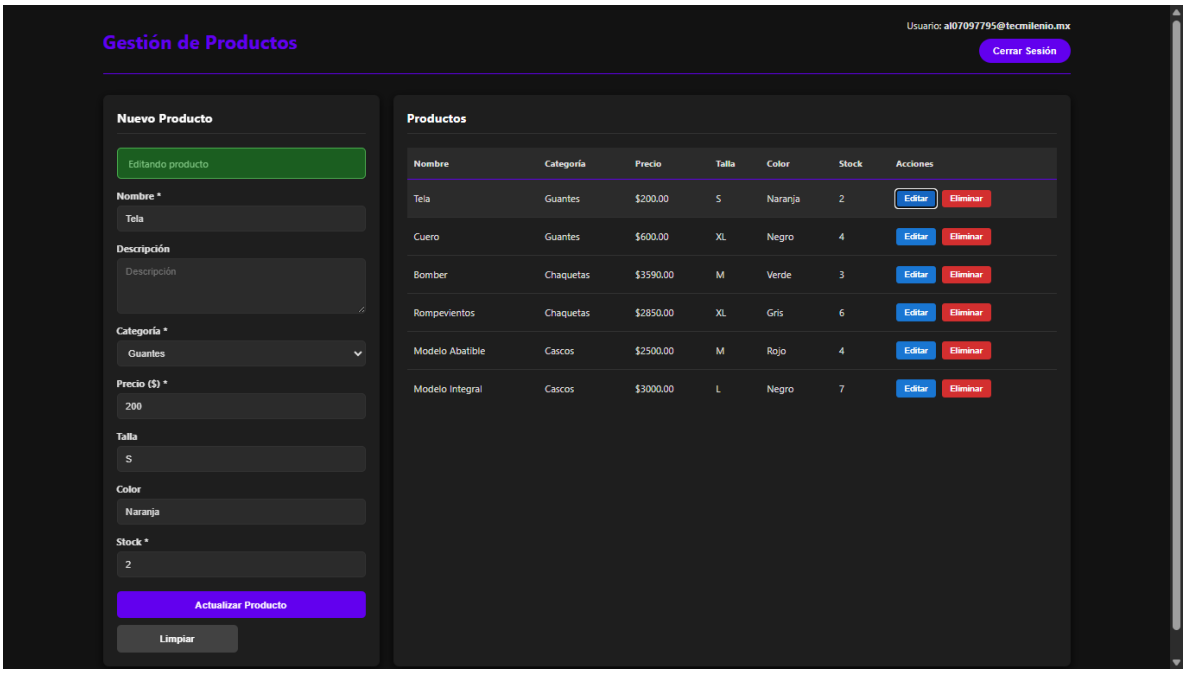
- TESTING.md (guía completa)
- TESTING_SUMMARY.md (resumen)
- TESTING_CHECKLIST.md (checklist)
- RESUMEN_PRUEBAS.md (guía rápida)

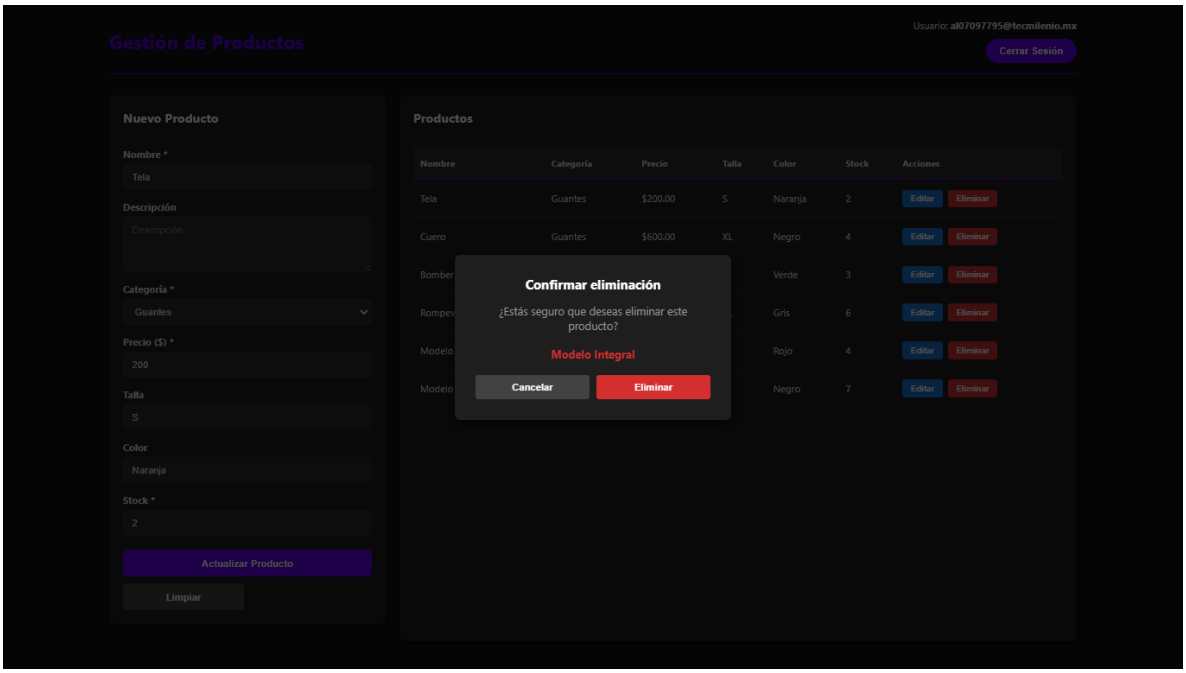
DESPLIEGUE


MOTO GEAR
Iniciar sesión
al07097795@tecmilenio.mx
••••••
Entrar
Registrarse


MOTO GEAR
Registro
Correo electrónico
Contraseña
Registrarse
Volver al login


Gestión de Productos
Usuario: al07097795@tecmilenio.mx
Cerrar Sesión
Nuevo Producto
Nombre *
Nombre del producto
Descripción
Descripción
Categoría *
-- Selecciona --
Precio ($) *
0.00
Talla
S, M, L, XL, etc.
Color
Color
Stock *
0
Guardar Producto
Limpiar
Productos
Nombre Categoría Precio Talla Color Stock Acciones
Tela Guantes $200.00 S Naranja 2 Editar Eliminar
Cuero Guantes $600.00 XL Negro 4 Editar Eliminar
Bomber Chaquetas $3590.00 M Verde 3 Editar Eliminar
Rompevientos Chaquetas $2850.00 XL Gris 6 Editar Eliminar
Modelo Abatible Cascos $2500.00 M Rojo 4 Editar Eliminar
Modelo Integral Cascos $3000.00 L Negro 7 Editar Eliminar


Gestión de Productos
Usuario: al07097795@tecmilenio.mx
Cerrar Sesión
Nuevo Producto
Editando producto
Nombre *
Tela
Descripción
Descripción
Categoría *
Guantes
Precio ($) *
200
Talla
S
Color
Naranja
Stock *
2
Actualizar Producto
Limpiar
Productos
Nombre Categoría Precio Talla Color Stock Acciones
Tela Guantes $200.00 S Naranja 2 Editar Eliminar
Cuero Guantes $600.00 XL Negro 4 Editar Eliminar
Bomber Chaquetas $3590.00 M Verde 3 Editar Eliminar
Rompevientos Chaquetas $2850.00 XL Gris 6 Editar Eliminar
Modelo Abatible Cascos $2500.00 M Rojo 4 Editar Eliminar
Modelo Integral Cascos $3000.00 L Negro 7 Editar Eliminar


Gestión de Productos
Usuario: al07097795@tecmilenio.mx
Cerrar Sesión
Nuevo Producto
Nombre *
Tela
Descripción
Descripción
Categoría *
Guantes
Precio ($) *
200
Talla
S
Color
Naranja
Stock *
2
Actualizar Producto
Limpiar
Productos
Nombre
Categoría
Precio
Talla
Color
Stock
Acciones
Tela
Guantes
$200.00
S
Naranja
2
Editar
Eliminar
Cuero
Guantes
$600.00
XL
Negro
4
Editar
Eliminar
Bomber
Verde
3
Editar
Eliminar
Rompev
Gris
6
Editar
Eliminar
Modelo
Rojo
4
Editar
Eliminar
Modelo
Negro
7
Editar
Eliminar
Confirmar eliminación
¿Estás seguro que deseas eliminar este producto?
Modelo Integral
Cancelar
Eliminar

Backend - Servidor y API
Se implementó un servidor con Node.js y Express que funciona como API RESTful. Este servidor realiza operaciones de base de datos MySQL (crear, leer, actualizar y eliminar registros). Se añadió autenticación mediante JSON Web Tokens (JWT) y se protegió la información con hashing de contraseñas usando bcryptjs.

Base de Datos
Se diseñó una base de datos MySQL con una tabla de usuarios que almacena nombre, correo electrónico, contraseñas protegidas y fecha de registro. La conexión se optimizó para manejar múltiples solicitudes simultáneas mediante un sistema de pooling.

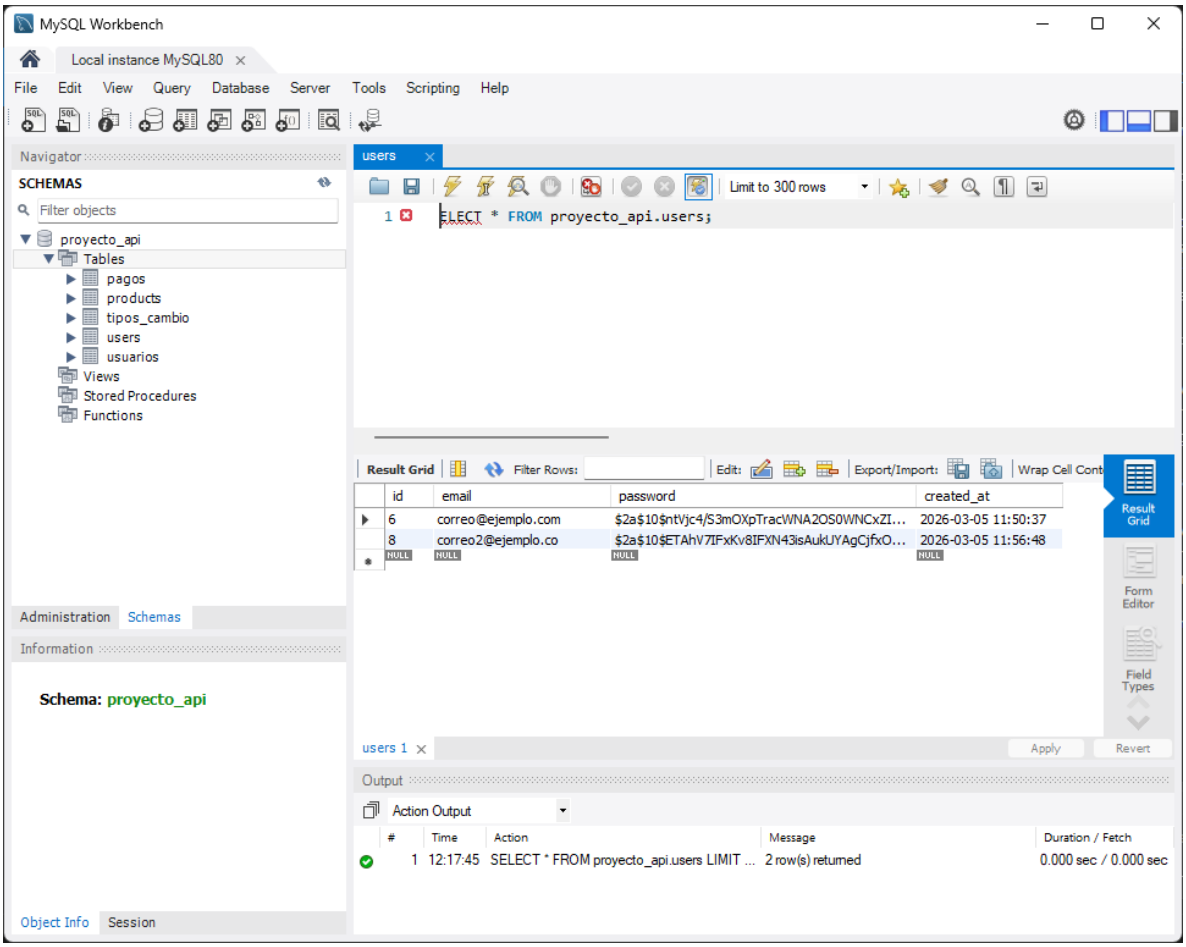
Frontend - Interfaz de Usuario
Se desarrollaron tres páginas principales: inicio de sesión, registro de usuarios y panel de control (dashboard). Estas páginas utilizan HTML para estructura, CSS para diseño visual y JavaScript para funcionalidades interactivas y comunicación con el backend.

Sistema de Autenticación y Navegación
El sistema verifica automáticamente el estado de autenticación del usuario. Si un usuario no logueado intenta acceder al panel de control, es redirigido al login. Si un usuario ya autenticado intenta acceder a login o registro, es redirigido directamente al dashboard.

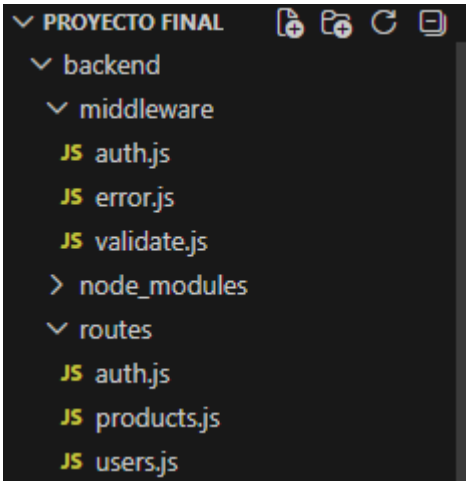
Arquitectura y Organización
El proyecto se estructuró en módulos separados por carpetas: configuración de base de datos, middlewares de autenticación, rutas de la API y archivos estáticos del frontend. Se utilizaron variables de entorno para manejar información sensible de manera segura.

Resultado Final
Se obtuvo una aplicación web funcional que integra frontend y backend, con autenticación segura, gestión de base de datos y una interfaz de usuario operativa. El sistema demuestra los principios fundamentales del desarrollo web moderno.

Base de Datos en MySQL



Librerias



Funcionalidades del Backend

```
const express = require('express');
const router = express.Router();
const { generateToken } = require('../middleware/auth');
const bcrypt = require('bcryptjs');
const { getPool } = require('../db');
const { AppError } = require('../middleware/error');
const { validate, validateAuthPayload } = require('../middleware/validate');
```

backend > middleware > JS validate.js > ...

```
const { AppError } = require('./error');

function validate(schemaFn) {
    return (req, res, next) => {
        const errors = schemaFn(req);
        if (errors.length > 0) {
            return next(new AppError(errors.join(' | '), 400));
        }
        return next();
    };
}

function validateAuthPayload(req) {
    const { email, password } = req.body;
    const errors = [];

    if (!email || typeof email !== 'string' || !email.includes('@')) {
        errors.push('Email valido es requerido');
    }

    if (!password || typeof password !== 'string' || password.length < 4) {
        errors.push('Password es requerida y debe tener al menos 4 caracteres');
    }

    return errors;
}
```

backend > middleware > JS auth.js > ...

```javascript
const jwt = require('jsonwebtoken');
const { AppError } = require('./error');

const JWT_SECRET = process.env.JWT_SECRET || 'your_jwt_secret_key_change_in_production';

// Middleware para proteger rutas
const authMiddleware = (req, res, next) => {
    try {
        const token = req.headers.authorization?.split(' ')[1];

        if (!token) {
            return next(new AppError('Token no proporcionado', 401));
        }

        const decoded = jwt.verify(token, JWT_SECRET);
        req.user = {
            id: decoded.id,
            email: decoded.email,
            role: decoded.role || 'user'
        };
        next();
    } catch (err) {
        return next(new AppError('Token invalido o expirado', 401));
    }
```

CRUD

```
// @route   GET /api/products
// @desc    Obtener todos los productos del usuario
// @access  Private
router.get('/', validate(validatePaginationQuery), async (req, res, next) => {
    try {
        const pool = getPool();
        const page = Number(req.query.page || 1);
        const limit = Number(req.query.limit || 10);
        const offset = (page - 1) * limit;

        const { whereClause, params } = buildProductsWhereClause(req.query, req.user);

        const [countRows] = await pool.query(
            `SELECT COUNT(*) AS total FROM products ${whereClause}`,
            params
        );

        const [rows] = await pool.query(
            `SELECT * FROM products ${whereClause} ORDER BY created_at DESC LIMIT ? OFFSET ?`,
            [...params, limit, offset]
        );

        const total = Number(countRows[0]?.total || 0);

        res.json({
            data: rows.map(mapProduct),
            pagination: {
                page,
                limit,
                total,
                totalPages: Math.ceil(total / limit) || 1
            },
            filters: {
```

```
// @route   GET /api/products/:id
// @desc    Obtener un producto específico
// @access  Private
router.get('/:id', async (req, res, next) => {
    try {
        const pool = getPool();
        const [rows] = await pool.query('SELECT * FROM products WHERE id = ? LIMIT 1', [
            req.params.id
        ]);

        if (rows.length === 0) {
            return next(new AppError('Producto no encontrado', 404));
        }

        const product = rows[0];

        if (req.user.role !== 'admin' && String(product.usuario_id) !== String(req.user.id)) {
            return next(new AppError('No tienes permiso para acceder a este producto', 403));
        }

        res.json(mapProduct(product));
    } catch (err) {
        next(err);
    }
});
```

```
// @route   POST /api/products
// @desc    Crear nuevo producto
// @access  Private
router.post('/', validate(validateProductPayload), async (req, res, next) => {
    try {
        const { nombre, descripcion, categoria, precio, talla, color, stock, status } = req.body;
        const pool = getPool();

        const [insertResult] = await pool.query(
            `INSERT INTO products (nombre, descripcion, categoria, precio, talla, color, stock, status, usuario
             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`,
            [
                nombre,
                descripcion || '',
                categoria,
                precio,
                talla || '',
                color || '',
                stock || 0,
                status || 'active',
                req.user.id
```

```
// @route   PUT /api/products/:id
// @desc    Actualizar producto
// @access  Private
router.put('/:id', validate(validateProductUpdatePayload), async (req, res, next) => {
    try {
        const pool = getPool();
        const [existingRows] = await pool.query('SELECT * FROM products WHERE id = ? LIMIT 1', [
            req.params.id
        ]);

        if (existingRows.length === 0) {
            return next(new AppError('Producto no encontrado', 404));
        }

        const existingProduct = existingRows[0];
        if (req.user.role !== 'admin' && String(existingProduct.usuario_id) !== String(req.user.id)) {
            return next(new AppError('No tienes permiso para editar este producto', 403));
        }

        const { nombre, descripcion, categoria, precio, talla, color, stock, status } = req.body;

        await pool.query(
            `UPDATE products
             SET nombre = ?, descripcion = ?, categoria = ?, precio = ?, talla = ?, color = ?, stock = ?, status
             WHERE id = ?`,
```

```
// @route   DELETE /api/products/:id
// @desc    Eliminar producto
// @access  Private
router.delete('/:id', async (req, res, next) => {
    try {
        const pool = getPool();
        const [rows] = await pool.query('SELECT * FROM products WHERE id = ? LIMIT 1', [
            req.params.id
        ]);

        if (rows.length === 0) {
            return next(new AppError('Producto no encontrado', 404));
        }

        const product = rows[0];

        if (req.user.role !== 'admin' && String(product.usuario_id) !== String(req.user.id)) {
            return next(new AppError('No tienes permiso para eliminar este producto', 403));
        }

        await pool.query('DELETE FROM products WHERE id = ?', [req.params.id]);

        res.json({ message: 'Producto eliminado correctamente' });
    } catch (err) {
        next(err);
```

Autenticación

```
// @route   POST /api/auth/register
// @desc    Registrar nuevo usuario
// @access  Public
router.post('/register', validate(validateAuthPayload), async (req, res, next) => {
    try {
        const { email, password } = req.body;
        const pool = getPool();

        const [existingUsers] = await pool.query(
            'SELECT id FROM users WHERE email = ? LIMIT 1',
            [email]
        );

        if (existingUsers.length > 0) {
            return next(new AppError('El usuario ya existe', 400));
        }

        const hashedPassword = await bcrypt.hash(password, 10);

        const [insertResult] = await pool.query(
            'INSERT INTO users (email, password, role) VALUES (?, ?, ?)',
            [email, hashedPassword, 'user']
        );

        const userId = insertResult.insertId;

        const token = generateToken(String(userId), email, 'user');
```

Framework obligatorio Angular

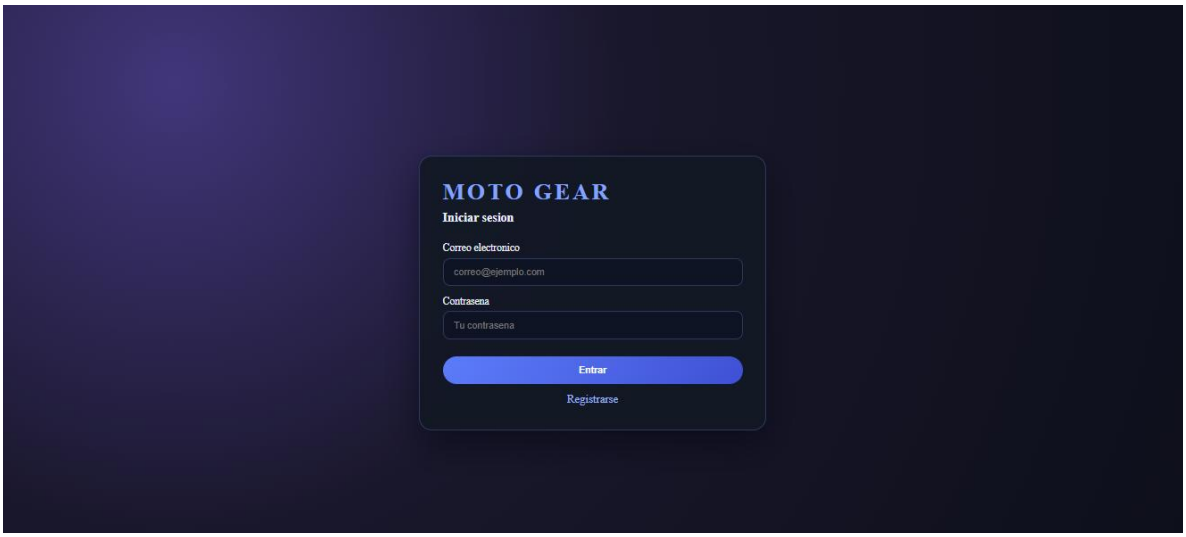
```
import { Routes } from '@angular/router';
import { authGuard } from './guards/auth.guard';
import { LoginPage } from './pages/login/login.page';
import { ProductsPage } from './pages/products/products.page';
import { RegisterPage } from './pages/register/register.page';
import { AdminUsersPage } from './pages/admin-users/admin-users.page';

export const routes: Routes = [
    { path: '', pathMatch: 'full', redirectTo: 'login' },
    { path: 'login', component: LoginPage },
    { path: 'register', component: RegisterPage },
    { path: 'products', component: ProductsPage, canActivate: [authGuard], data: { roles:
    { path: 'admin/users', component: AdminUsersPage, canActivate: [authGuard], data: { r
    { path: '**', redirectTo: 'login' }
```

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideHttpClient } from '@angular/common/http';
import { provideRouter } from '@angular/router';

import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideHttpClient(),
    provideRouter(routes)
```

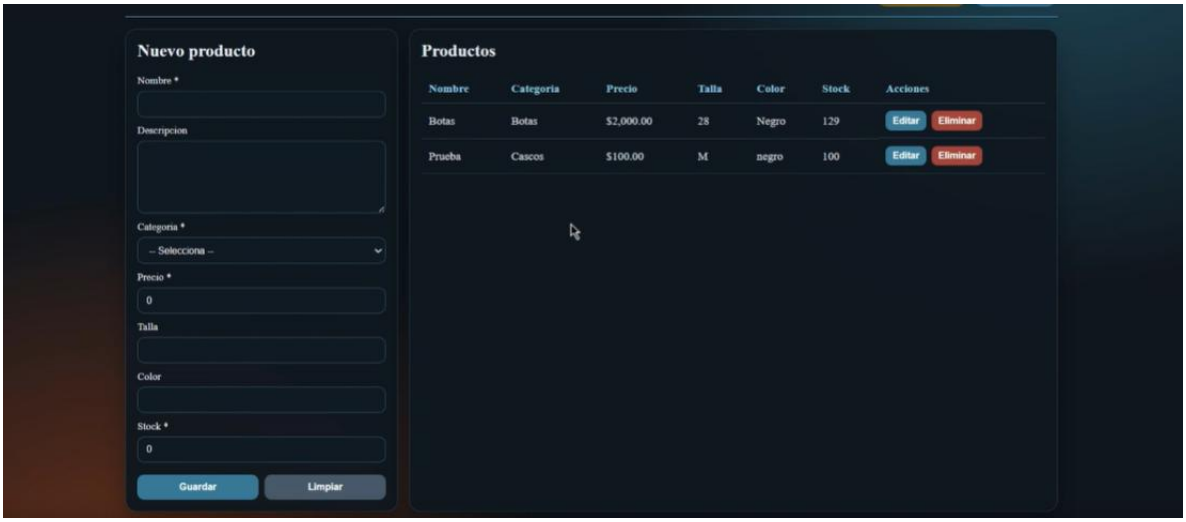
```
import { CommonModule } from '@angular/common';
import { Component, OnDestroy, OnInit } from '@angular/core';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { finalize, retry, timeout } from 'rxjs/operators';
import { UserRecord } from '../../models/user.model';
import { AuthService } from '../../services/auth.service';
import { UsersService } from '../../services/users.service';

@Component({
  selector: 'app-admin-users-page',
  imports: [CommonModule, ReactiveFormsModule],
  templateUrl: './admin-users.page.html',
  styleUrl: './admin-users.page.css'
})
export class AdminUsersPage implements OnInit, OnDestroy {
  private readonly usersCacheKey = 'adminUsersCache';
  private readonly autoRefreshMs = 5000;
  private refreshIntervalId: ReturnType<typeof setInterval> | null = null;
  private requestInFlight = false;

  users: UserRecord[] = [];
  loading = false;
  loadError = false;
  message = '';
  messageType: 'success' | 'error' = 'success';
  editingUserId: string | null = null;
```

Requisitos mínimos del frontend

Pantalla de Login



Funcionalidades



Estructura Obligatoria del Repositorio

Este proyecto es una aplicacion full stack para gestionar productos con autenticacion de usuarios.
Tiene dos partes principales: un backend en Node.js con Express y MySQL, y un frontend en Angular.

En el backend se maneja toda la logica de la API.
Hay rutas para registro e inicio de sesion, rutas para CRUD de productos y rutas para administrar usuarios.
La seguridad se hace con JWT, por lo que las rutas privadas validan el token antes de responder.
La conexion a MySQL esta centralizada en db.js y al iniciar el servidor se crean las tablas necesarias si no existen.

La base de datos tiene principalmente dos tablas: users y products.
Users guarda correo, contrasena encriptada y fecha de creacion.
Products guarda nombre, descripcion, categoria, precio, talla, color, stock y relacion con el usuario que lo creo.

En el frontend Angular hay paginas de login, registro, productos y administracion de usuarios.
Los servicios consumen la API del backend y envian el token en los headers para acceder a rutas protegidas.
El guard de rutas evita entrar a paginas privadas sin sesion activa.

El flujo funcional es: el usuario se registra o inicia sesion, accede a la pantalla de productos para crear, editar y eliminar registros, y desde ahi puede entrar al modulo de administracion para ver correos de usuarios registrados y gestionarlos.

Durante la mejora del proyecto se corrigieron problemas de visualizacion de datos que mostraban tablas vacias aunque la base tuviera informacion.
Se robustecio el consumo de API, se diferencio mejor entre estados de carga, error y lista vacia, se agrego cache local para que no desaparezcan datos al refrescar y se implemento actualizacion automatica periodica para ver cambios sin recargar manualmente.

En resumen, es un sistema completo con autenticacion, persistencia en MySQL, panel de productos y modulo de administracion, con enfoque en estabilidad de datos y buena experiencia de uso.